

I. Milic & C. J. Díaz Baso • GPL-3.0 • github.com/ivanzmilic/cloudtorch • described at commit 1b10fab + uncommitted change (source-function penalty $10^3 \rightarrow 10^6$) • 24 code cells, no markdown • kernel pt, Python 3.12

1. WHAT IT DOES

Inverts one time step of an H α imaging-spectroscopy cube of a solar filament. Each pixel's spectrum is modelled as *two absorbing slabs* ("clouds") stacked along the line of sight on top of a background profile that is *not* fitted but supplied from the data. All 15 080 pixels are inverted *simultaneously* in a single PyTorch graph, with spatial-regularization terms coupling neighbouring pixels — the point of the port from the older `scipy+MPI` per-pixel code in `original_cloud_model/`.

The two slabs are initialised at -20 and $+20 \text{ km s}^{-1}$ to capture the counter-streaming blue/red-shifted components of filament threads.

2. DATA

Two `.npz` archives in `/dat/milic/MiHI_filament/` (paths hardcoded; FITS paths in the code are commented out):

- `mihi_all_data.npz` (5.8 GB) — data (113,146,160,550) float32 = (t, y, x, λ); wav (550,).
- `mihi_rvm_reconstruction_averages_skglm.npz` (11.6 GB) — `new_fit`, `new_extrafit`, same shape.

Line. $\lambda \in [6560.42, 6565.09] \text{ \AA}$, $\Delta\lambda = 8.5 \text{ m\AA}$ — H α 6562.8 $\text{\AA}$. Instrument: MiHI (SST). Intensity only, no polarimetry.

Selection. Time step `t=13`; spatial crop `[15:-15,15:-15]`; λ slice `[100:500]`. Result: `tofit` of shape **(116, 130, 400)** = 6032 000 data points, covering 6561.27–6564.66 $\text{\AA}$ ($\pm 77 \text{ km s}^{-1}$).

Normalisation. Divided by `Iqs=mean(tofit[:, :, -20:]) = 0.7584`, a pseudo-continuum from the 20 reddest points; the background is divided by the same constant.

Background. `bdata = rdata - restra`. The reconstruction decomposes every spectrum with `skglm.Lasso` on a dictionary of negative Gaussians plus a continuum term; `new_extrafit` is the Gaussian-only (extra absorption) part, so the difference isolates the smooth quiet-Sun profile behind the filament — exactly the incident intensity I_{in} the cloud model needs.

3. FORWARD MODEL

`cloud_model_torch.model_synth_2clouds_givenbck`. Each slab applies isothermal slab transfer,

$$I_{\text{out}}(\lambda) = I_{\text{in}}(\lambda) e^{-\tau_\lambda} + S(1 - e^{-\tau_\lambda}),$$

$$\tau_\lambda = |\Delta\tau| H(a, x), \quad x = \frac{\lambda - \lambda_0(1 + v/c)}{\Delta\lambda_D},$$

$$\Delta\lambda_D = \frac{|\Delta v|}{c} \lambda_0 + 10^{-2} \text{ \AA}.$$

Cloud 1 acts on the background, cloud 2 on cloud 1's output. H is a Humlíček-style complex rational approximation (`fvoigt`) evaluated in `torch` complex arithmetic, so the whole model is autograd-differentiable. λ_0 is taken as the *window mean*, 6562.9663 $\text{\AA}$.

Parameter vector, `(n_pixels, 10)`:

$$\mathbf{p} = [S, \Delta\tau, v_{\text{los}}, \Delta v, \log a]_{c1} \oplus [\dots]_{c2}$$

The `me()` Milne-Eddington routine and the 11/16-parameter entry points in the same module are *not* used here — the background replaces them.

4. INVERSION

Each physical quantity is a separate `nn.Parameter` of shape (15080,), restacked every iteration by `combine_parameters()`. This is what "mparams" means, and it is what allows individual quantities to be frozen: **both** `log a` are fixed ($-4, -5$) because free damping produced NaNs. Free parameters: $8 \times 15\,080 = 120\,640$, i.e. a 50:1 data-to-parameter ratio.

Loss. `torch.mean(torch.sum((obs-syn)**2))` — an unweighted, unnormalised sum of squared residuals (the outer mean over a scalar is a no-op; there is no noise weighting).

Penalties (14 terms, from `utils.py`); `regu2` is an L2 roughness penalty against a 3×3 reflection-padded neighbour average, `regu_min/regu_max` are one-sided ReLU hinges:

Term	Applies to	Weight
<code>regu2</code>	S_1, S_2 smoothness	10^6
<code>regu2</code>	$\Delta\tau$ smoothness	10^{-1}
<code>regu2</code>	v_{los} smoothness	10^{-2}
<code>regu2</code>	Δv smoothness	10^{-2}
<code>regu_min</code>	$\Delta v \geq 8 \text{ km s}^{-1}$	10^8
<code>regu_min</code>	$S, \Delta\tau \geq 0$	10^7
<code>regu_max</code>	$S \leq 0.6$	10^7

The 8 km s^{-1} floor is justified in-notebook by the thermal speed of hydrogen at 5000 K, $\sqrt{2k_B T/m_H} = 9.08 \text{ km s}^{-1}$.

Optimizer. Adam, `lr=5e-2`, **5000 iterations**, full field per step (no batching, no LR schedule, no early stopping — only a NaN guard that dumps the offending parameter rows).

Initial values. $S = 0.2/0.6$, $\Delta\tau = 4.5$, $v = \mp 20$, $\Delta v = 12$ for clouds 1/2.

Recorded run. `cuda`, 5000 iterations in 1:13 (68 it/s); final loss=1.02e5, chi2loss=2.79e3, reguloss=9.93e4 — the penalty term now dominates χ^2 by $\sim 35\times$. Mean $\log_{10} \chi^2 = -3.36$. Raising the S penalty from 10^3 to 10^6 cost only $\sim 8\%$ in χ^2 while flattening the S maps almost completely, which is the experiment the current working tree encodes.

5. OUTPUTS

Written to `figs_S_reg_1E6/` (untracked). **No arrays are saved** — `parameters` and `fit_spectra` live only in the kernel session.

- `inversion_maps.png` — 2×4 maps of the 8 free parameters.
- `fit_comparison_random_profiles.png` — 3 random pixels, data vs. fit.
- `fit_comparison_three_wavelength_maps.png` — observed vs. fitted images at 10/50/90% of the window.
- `fit_chisq_map.png` — $\log_{10} \chi^2$ map.
- `inversion_results_velocities*.png` — velocities masked by $\frac{1}{2}[1 + \tanh k(\tau - 1.5)]$ and weighted by $S(1 - e^{-\tau})$, suppressing velocities where the slab is optically thin and therefore unconstrained.

6. CAVEATS

- λ_0 is the window mean (6562.9663 $\text{\AA}$), not the H α rest wavelength — a $+7.2 \text{ km s}^{-1}$ shift of the velocity zero point unless wav already accounts for it.
- The $+10^{-2} \text{ \AA}$ Doppler-width epsilon (added to stop the Voigt profile exploding) bakes $\approx 0.46 \text{ km s}^{-1}$ of spurious width into every reported Δv .
- Positivity comes from `torch.abs()` inside the forward model, which makes the likelihood exactly symmetric about zero (a sign degeneracy) and puts a gradient kink at 0; the hinge penalties are the real bounds.
- `regu2` reshapes with `img.shape[0:2]` while every other `regu*` uses `img.shape[1:3]`, so they must be handed differently shaped arrays. The notebook does this correctly, but it is a live trap.
- Returned `syn` is from *before* the last `optimizer.step()` while `final_pams` is from after it — a one-step mismatch.
- `print(np.mean(chisq[0,0]))` reports one pixel, not the field mean.
- Hard-coded machine-specific data paths; `optimization()` is duplicated across the notebooks and scripts; cell 23 is dead debug code.